

ZHI-WEN TING

Firmware / System Software Portfolio

GitHub | github.com/copywen-8763 | Email

M.S. in Electrical Engineering

National Taipei University of Technology

Core Skills

C / C++ | Linux | STM32 | ESP32 | FreeRTOS | UART / SPI / I2C | Git / GDB

Focused on embedded systems, low-level software, and hardware-software integration. Experience includes embedded MCU development, Linux C programming, RISC-V system emulation, hardware-software integration, and embedded communication.

Smart Traffic Signal System

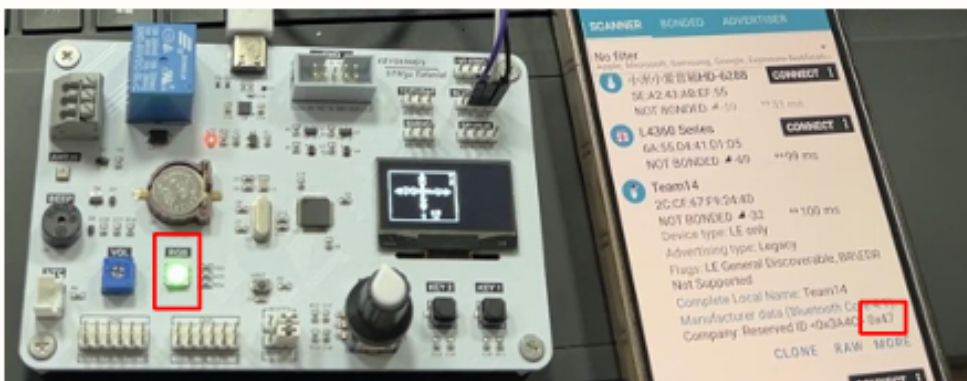
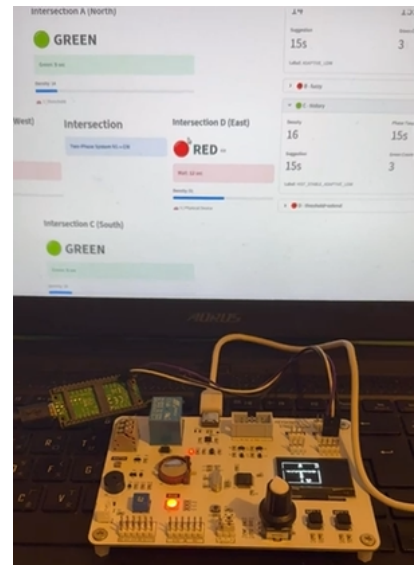
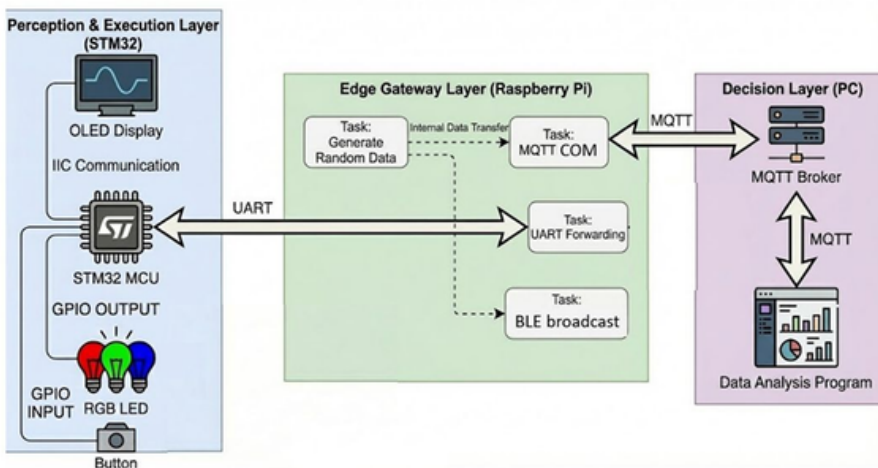
STM32 | Raspberry Pi | UART | I2C | MQTT | BLE

Overview

Developed a distributed smart traffic-control prototype integrating an STM32-based intersection controller, a Raspberry Pi communication gateway, and a PC-based decision system.

Implementation

- Developed STM32 peripheral control for traffic lights, buttons, and OLED display using GPIO and I2C.
- Implemented UART communication between STM32 and Raspberry Pi for command and status exchange.
- Used MQTT over Wi-Fi for communication between Raspberry Pi and the PC-based decision system.
- Implemented BLE broadcasting for nearby status notification.
- Integrated the embedded controller, communication gateway, and PC decision layer into an end-to-end prototype.



NEMU RISC-V Emulator — NJU-ICS

C | Linux | RISC-V | Computer Systems

Overview

Extended the NEMU RISC-V emulator through NJU ICS 2020 PA0–PA2 to explore computer architecture, instruction execution, and low-level system behavior.

Implementation Highlights

- Implemented a simple debugger with expression evaluation and watchpoints for monitoring program execution.
- Implemented and debugged RISC-V instruction decoding and execution to support C programs and CPU test cases.
- Worked with register state, physical memory access, and the instruction execution loop to understand the interaction between ISA and emulator architecture.
- Verified emulator behavior by running test programs and applications under NEMU.

```
Reusing isolated NEMU build
Reusing isolated AM game image
AM game ready: 171860508
Image: /mnt/c/Users/copywen/Documents/ChatGPT/FW_L/tmp/nes-runtime
emu.bin
[src/utils/log.c:30 init_log] Log is written to stdout
[src/memory/paddr.c:50 init_mem] physical memory area [0x80000000
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'serial' at [0x
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'rtc' at [0xa
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'vgactl' at [0
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'vmem' at [0x
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'keyboard' at
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'audio' at [0
[src/device/io/mmio.c:50 add_mmio_map] Add mmio map 'audio-sbuf'
[src/monitor/monitor.c:60 load_img] The image is /mnt/c/Users/cop
ection/171860508/build/171860508-riscv32-nemu.bin, size = 17348
[src/monitor/monitor.c:28 welcome] Trace: OFF
[src/monitor/monitor.c:32 welcome] Build time: 18:30:11, Aug 22 2
Welcome to riscv32-NEMU!
For help, type "help"
Welcome to TETRIS!
Press SPACEBAR to start!
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [SPIN]
```

```
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVL]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVL]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVL]
[Log][171860508/main.c,469,Wv1JqEgtYr] [FALL]
[Log][171860508/main.c,542,Ni4LED3JUH] Touch ground
[Log][171860508/main.c,551,Ni4LED3JUH] New tetromino: ((7
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [FALL]
[Log][171860508/main.c,542,Ni4LED3JUH] Touch ground
[Log][171860508/main.c,587,wcwGFnsChC] Scored: 500
[Log][171860508/main.c,551,Ni4LED3JUH] New tetromino: ((7
[Log][171860508/main.c,469,Wv1JqEgtYr] [SPIN]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,469,Wv1JqEgtYr] [MOVR]
[Log][171860508/main.c,542,Ni4LED3JUH] Touch ground
[Log][171860508/main.c,551,Ni4LED3JUH] New tetromino: ((7, 2), 11)
```



Image Processing Algorithms from Scratch

C | Linux | Performance Analysis

Overview

Implemented a collection of image-processing algorithms from scratch in C on Linux without relying on OpenCV, with emphasis on algorithm implementation and runtime behavior.

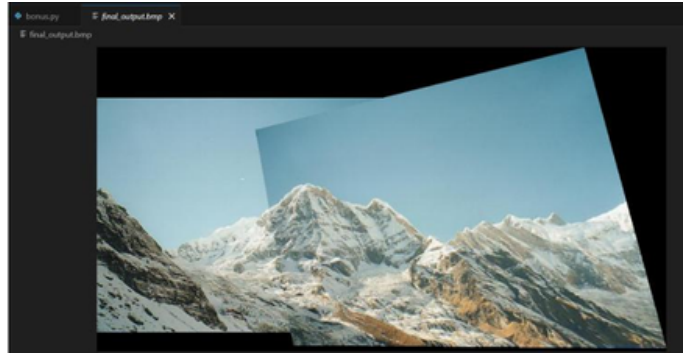
Implementation

- Implemented BMP parsing, image rotation, resizing, color conversion, and basic image manipulation.
- Implemented binarization, morphology, and connected-component labeling.
- Calculated region area, centroid coordinates, and image orientation.
- Implemented Inverse Perspective Mapping (IPM) and inverse warping for bird's-eye-view transformation.
- Developed horizon detection and ROI preprocessing mechanisms.

Performance Analysis

- Compared execution time between the C implementation and Python open-source libraries to examine algorithmic efficiency and runtime performance.

GitHub:[cbmp_processor](https://github.com/copywen/cbmp_processor)|[computer-vision-assignments](https://github.com/copywen/computer-vision-assignments)



Other Technical Projects

FPGA Digital Design & Verification RTL | FPGA | Digital IC Design

Overview

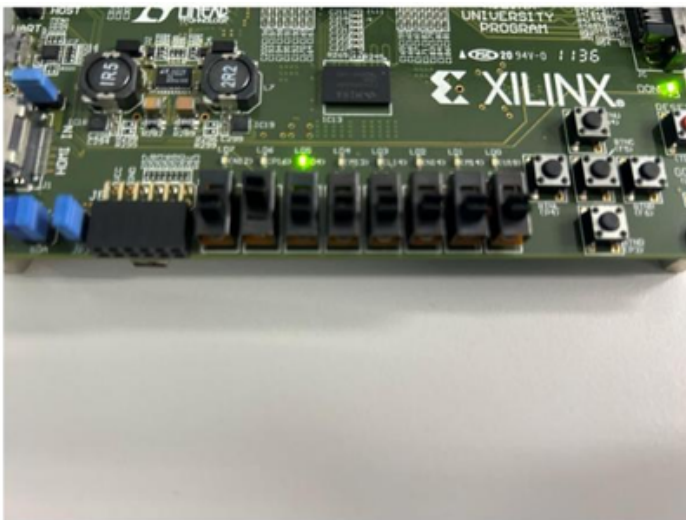
Implemented and verified a digital arithmetic design on an FPGA platform as part of an advanced digital IC design project.

Implementation

- Studied a digital architecture containing a processing-element array larger than 4×4 .
- Collaborated on the implementation of a multiplier-based circuit for cryptographic computation.
- Configured design parameters and verified functional behavior through FPGA implementation.
- Compared FPGA output against manually calculated expected results to validate correctness.

Result

- Successfully completed hardware-level functional verification on the FPGA development board



Ship Wake Segmentation & Orientation Analysis

Python | U-Net | Unity | Computer Vision

Overview

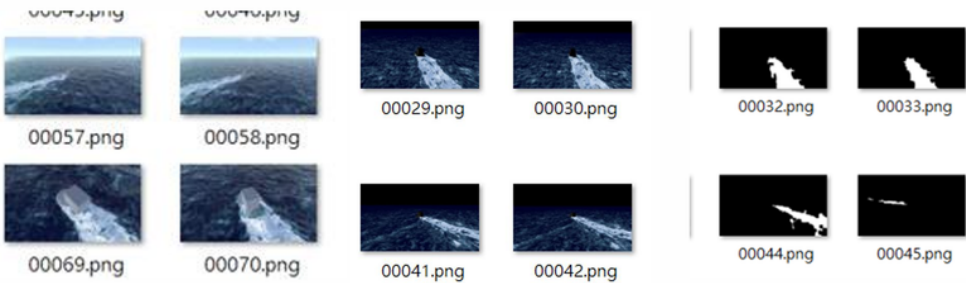
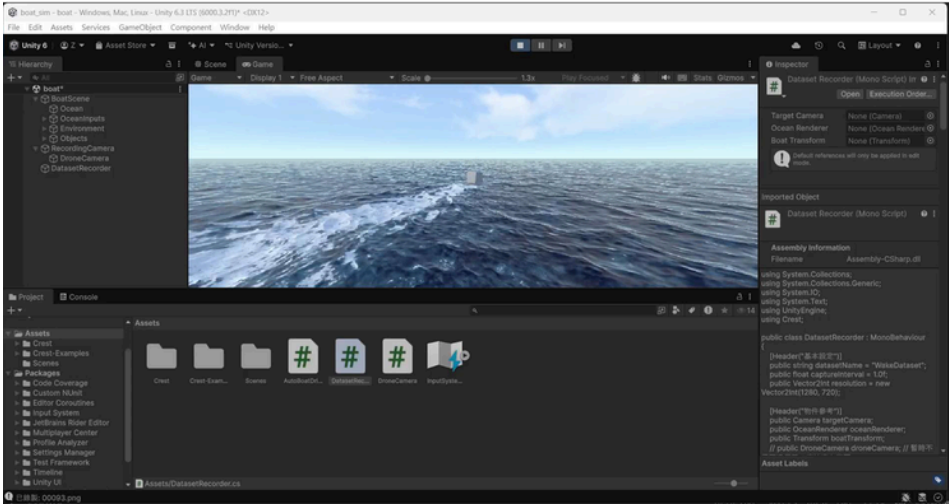
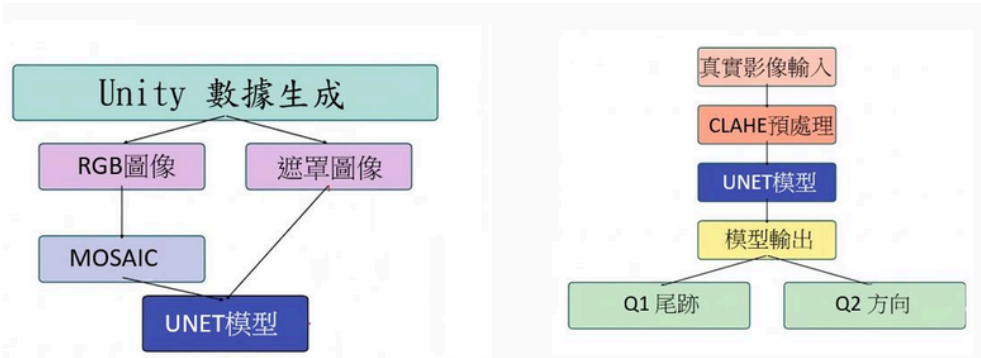
Developed a computer-vision pipeline for detecting ship wakes and estimating vessel orientation from maritime imagery.

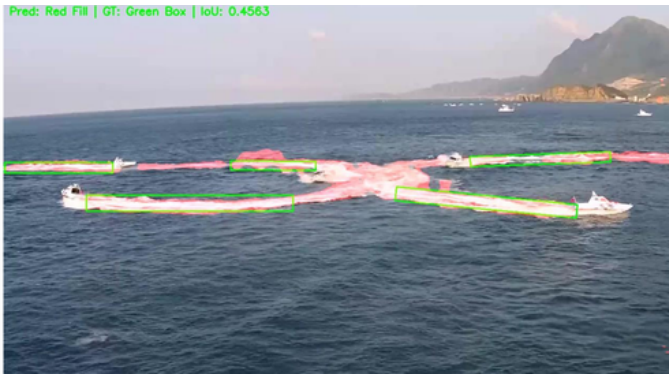
Synthetic Dataset Generation

- Built a Unity-based simulation environment and developed C# scripts to generate synthetic maritime images.
- Generated corresponding segmentation masks by disabling lighting during rendering, enabling automated ground-truth generation.
- Applied Mosaic augmentation to improve robustness for scenes containing multiple targets.

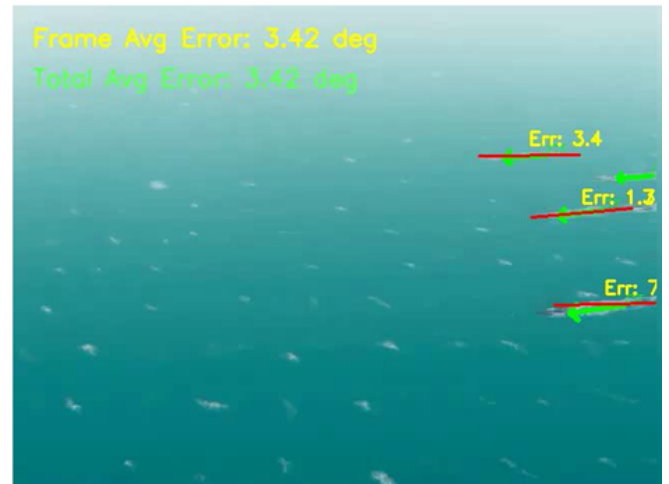
Model & Analysis

- Trained a U-Net segmentation model to identify ship wakes from water-surface images.
- Post-processed segmentation results to estimate wake orientation and vessel heading.
- Evaluated the pipeline on video data, achieving an average orientation error of approximately 5.5°.





Wake segmentation and vessel orientation estimation results



Example orientation error visualization

IC / SoC Design

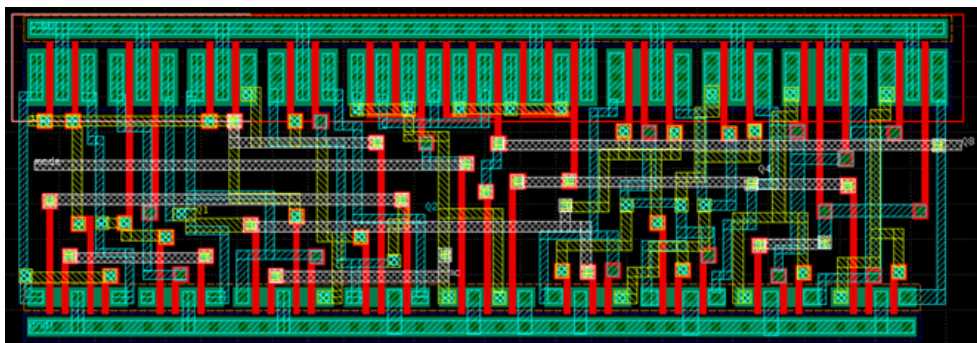
RTL Comparator Design

- Implemented two functionally equivalent comparator architectures in RTL.
- Synthesized both designs and compared their area and power characteristics to evaluate architectural trade-offs.

Number of ports:	727	Number of ports:	850
Number of nets:	1461	Number of nets:	1748
Number of cells:	822	Number of cells:	1025
Number of combinational cells:	693	Number of combinational cells:	890
Number of sequential cells:	105	Number of sequential cells:	105
Number of macros/black boxes:	0	Number of macros/black boxes:	0
Number of buf/inv:	171	Number of buf/inv:	216
Number of references:	11	Number of references:	12
Combinational area:	15022.022474	Combinational area:	19239.897714
Buf/Inv area:	2128.896034	Buf/Inv area:	3243.240025
Noncombinational area:	7747.185883	Noncombinational area:	7973.381100
Macro/Black Box area:	0.000000	Macro/Black Box area:	0.000000
Net Interconnect area:	78727.065674	Net Interconnect area:	97860.495087
Total cell area:	22769.208357	Total cell area:	27213.278814
Total area:	101496.274031	Total area:	125073.773901

VLSI Layout — Divide-by-8/9 DMP Divider

- Designed the physical layout of a divide-by-8/9 DMP frequency divider using KLayout.
- Optimized placement and routing for reduced area while satisfying DRC and LVS verification requirements.



Leadership & Activities

Equipment Lead / Event Coordinator
Coordinated equipment and logistics
for a campus charity event with 100+
participants; also served as event host.



Technology R&D Club — Teaching Lead
Built Arduino-based prototypes and
mentored junior members in project
development.